

AHE-MRTA: Ekosistem-Güdümlü Paradigma Seçimi

Yöntemin Sade ve Ayrıntılı Anlatımı

AHE-MRTA Projesi — Teknik Doküman

June 21, 2026

Özet (bir paragrafta yöntem)

Birden çok robotu, gelen görevlere atamak istiyoruz. Görevler zaman içinde çıkıyor, robotlar bazen bozuluyor, bazı görevlerin son teslim zamanı (deadline) yaklaşıyor. **Tek bir atama kuralı her duruma uymaz:** en yakın robotu seçmek yoğunlukta hızlıdır ama deadline'ı kaçırabilir; deadline'a kilitlenmek arıza anında yavaş kalır; “kararı bir kez ver, bir daha değiştirme” yaklaşımı kararlıdır ama bozulan robotun işini kurtaramaz. **AHE-MRTA'nın fikri:** bu kuralları (paradigmaları) bir *ekosistemde* yarıştırmak ve her an, o anki duruma en uygun kuralı otomatik seçmek. Robotların ve görevlerin durumu dört sayıya (*işletim durum vektörü*) sıkıştırılır; bu dört sayı, beş “hormonun” (her biri bir atama kuralı) gücünü günceller; en güçlü hormon o anki kuralı seçer ve atamayı yapar. Böylece sistem, koşullar değiştikçe kural değiştirir. Tüm akış Şekil 1'dedir.

Contents

1	Yöntemin Mantığı: Neden “Ekosistem”?	1
2	Problemi Tanımlayalım	2
3	Yöntem: AHE-MRTA Adım Adım	2
3.1	Adım 1 — Bağlam Vektörü: “Durum nedir?”	2
3.2	Adım 2 — Ekosistem Dinamiği: Kuralların Yarışı	3
3.3	Adım 3 — Seçici: Hangi Kural Çalışsın?	4
3.4	Adım 4 — Kural Kütüphanesi: Beş Atama Kuralı	4
3.5	Tam Algoritma	5
3.6	Kararlılık: Gereksiz Değişimi Önlemek	5
3.7	Maliyet ve Sabit Ayarlar	6
4	Somut Örnek: Bir Robot Bozulduğunda	6

1 Yöntemin Mantığı: Neden “Ekosistem”?

Diyelim ki bir saha ekibini yönetiyorsunuz. Bazen “en yakın olan gitsin” dersiniz, bazen “en acil işi önce bitirelim”, bazen bir araç bozulunca “onun işlerini hemen başkalarına dağıtalım”. **Aynı kural her zaman doğru değildir; doğru kural duruma bağlıdır.** Literatürde buna *no-free-lunch* denir: hiçbir tek sezgisel (heuristic) tüm koşullarda en iyi olamaz.

AHE-MRTA tam da bunu yapar: birden çok atama kuralını elinde tutar ve **çalışma sırasında** aralarında geçiş yapar. Geçiş bir *biyomimetik ekosistem* ile yönetir — doğadaki gibi, koşullara en uygun “tür” (burada: atama kuralı) baskın hale gelir. Mekanizmanın üç parçası vardır:

1. **İşletim Durum Vektörü** $c(t) \in [0, 1]^4$ — “şu an durum nedir?” sorusunun dört sayılık cevabı: ne kadar yoğunuz, kaç robot uygun, deadline baskısı var mı, arıza var mı (Bölüm 3.1).
2. **Ekosistem dinamiği** $D(t) \in \mathbb{R}^5$ — beş kuralın (hormonun) o anki “gücünü” tutan ve işletim durumuna göre güncelleyen bir yarış (Bölüm 3.2).
3. **Seçici (dispatcher)** — en güçlü hormona karşılık gelen kuralı seçip çalıştırır; acil durumlarda (arıza/deadline) yarışı beklemeden doğrudan müdahale eder (Bölüm 3.3–3.4).

Sezgi. Ekosistem *atamayı kendisi çözmez*; sadece “şu an hangi klasik çözücü çalışsın?” sorusunu cevaplar. Bu yüzden yönteme *hiper-sezgisel* (hyper-heuristic) denir: kuralların üstünde, kural seçen bir katman.

Her kural (paradigma) AHE’nin *kendi* kodudur; rakip bir yönteme iş devredilmez. Kurallar arası tek “hafıza” baskınlık vektörü $D(t)$ ’dir.

2 Problemi Tanımlayalım

Elimizde ne var? $\mathcal{R} = \{r_1, \dots, r_n\}$ robotlar, $\mathcal{T}(t)$ ise t anında bekleyen (açık) görevler. Her görev τ şunları taşır: bir konum p_τ , bir son teslim zamanı (deadline) d_τ , bir öncelik $\pi_\tau \in \{1, 2, 3\}$ ve bir yetenek gereksinimi c_τ (görevi hangi tip robot yapabilir). Her robot r ’nin durumu $s_r(t) = (p_r, \tau_r^{\text{cur}}, \phi_r)$: konum, o an yaptığı görev ve canlı/arızalı bayrağı.

Ne karar veriyoruz? Her *karar anında* t_k (bir görev geldi, bir robot bozuldu, bir deadline yaklaştı veya periyodik zamanlayıcı ateşledi) hangi robotun hangi görevi yapacağını söyleyen bir atama $x_{r,\tau}(t_k) \in \{0, 1\}$ üretiriz. İki kısıt: her robotun kuyruğu en fazla Q görev tutar ($\sum_\tau x_{r,\tau} \leq Q$) ve robot, görevin yeteneğini karşılamalıdır.

Bir atama ne kadar “iyi”? Her robot–görev çiftine bir *maliyet* biçeriz; düşük maliyet = iyi eşleşme. Maliyet dört şeyi birleştirir: yol süresi $T_{r,\tau}$ (uzak robot pahalı), aciliyet $u_\tau = (d_\tau - t_k)^{-1}$ (deadline yakınsa öne al), kritiklik π_τ (yüksek öncelik önce) ve yeniden-atama cezası $\rho_{r,\tau}$ (gereksiz yere atamayı değiştirme):

$$C_{r,\tau} = w_d T_{r,\tau} + w_u u_\tau^{-1} + w_\pi \pi_\tau^{-1} + w_\rho \rho_{r,\tau}. \quad (1)$$

Buradaki ağırlıklar $w = (w_d, w_u, w_\pi, w_\rho)$ **sabit değildir**; o an seçilen kural (paradigma) belirler (Bölüm 3). Yani aynı maliyet formülü, farklı ağırlıklarla farklı “kişilikler” kazanır.

Sezgi. Aciliyet terimi küçük bir incelik taşır: bir görev zaten doğru robotun kuyruğunda ve zamanında gidecekse, aciliyeti *şışirmeyiz*. Böylece yaklaşan bir deadline yalnızca *atanmamış* veya risk altındaki görevleri öne çeker; hâlihazırda yolunda olan atamaları boş yere savurmaz. Bu, gereksiz atama değişimini (churn) frenler.

Neyi ölçüyoruz? Koşu boyunca dokuz metrik raporlanır: tamamlama oranı (CR), makespan, ortalama gecikme, deadline ihlal oranı (DVR), arıza kurtarma süresi, Jain iş-yükü dengesi, yürütme preempt sayısı, yeniden-dağıtım oranı ve karar gecikmesi. CR ve DVR birincildir.

Sezgi. **Dürüst ölçüm kuralı:** Gecikme ve DVR yalnız *tamamlanan* görevlerden değil, *istenen tüm* görevlerden hesaplanır. Bir yöntem zor görevleri sessizce bırakırsa, normalde gecikmesi “yapay olarak” düşük görünürdü. Biz bitmeyen görevi ufukta sansürleyip (gecikme) ve deadline ihlali sayıp (DVR) bu hileyi engelleriz. Bu kural *tüm* yöntemlere aynen uygulanır.

Son olarak ortam *çevrimiçidir*: görevler akarken, robotlar her an bozulabilirken, her karar olay başına 50 ms’lik bir bütçede verilmelidir.

3 Yöntem: AHE-MRTA Adım Adım

3.1 Adım 1 — İşletim Durum Vektörü: “Durum nedir?”

Sezgi. Sistem karar vermeden önce, o anki durumu *dört sayıya* özetler. Bu dört sayı, “hangi kural şu an işe yarar?” sorusunun ipuçlarıdır.

Table 1: Her kuralın “en uygun olduğu durum” profili $V_i \in [0, 1]^4$. Sütunlar: td = görev-yoğunluğu, ra = robot-uygunluğu, dp = deadline-baskısı, fr = arıza-oranı. Yüksek değer: bu boyut yüksekken kural etkili.

Hormon (kural)	td	ra	dp	fr
H_SPATIAL (en yakın)	0.7	0.7	0.1	0.1
H_CRIT (öncelik önce)	0.3	0.5	0.8	0.2
H_TEMP (deadline, varsayılan)	0.5	0.5	0.9	0.1
H_STAB (kararı koru)	0.3	0.3	0.3	0.8
H_RECOV (arıza kurtar)	0.3	0.2	0.2	0.9

n robot sayısı, m açık görev sayısı, $\mathcal{R}^{\text{av}}$ uygun robotlar (canlı, takılı değil, yer var) ve $\mathcal{R}^f$ arızalı/takılı robotlar olsun. Dört bileşen:

$$c_1 = \min(1, m/n), \quad \text{görev yoğunluğu (kaç iş düşüyor)} \quad (2)$$

$$c_2 = |\mathcal{R}^{\text{av}}|/n, \quad \text{robot uygunluğu (eldeki güç)} \quad (3)$$

$$c_3 = |\{\tau : d_\tau - t_k \leq \Delta_d\}| / \max(1, m), \quad \text{deadline baskısı (acil iş oranı)} \quad (4)$$

$$c_4 = |\mathcal{R}^f|/n, \quad \text{arıza oranı (bozulma)} \quad (5)$$

deadline ufkü $\Delta_d = 60$ s. Her sayı $[0, 1]$ aralığındadır. Anlamları: c_1, c_2 “ne kadar yük, ne kadar güç” dengesini; c_3 “ne kadar acele” gerektiğini; c_4 “ne kadar bozulma” olduğunu söyler. Dördü de, robotların yayınladığı durum özetlerinden hesaplanır — robotlar arası ekstra iletişim gerekmez.

Her kuralın (hormonun) bir “en sevdiği durum” profili vardır (Tablo 1). Örneğin arıza kurtarma kuralı (H_RECOV), arıza oranı (fr) yüksekken parlar; deadline kuralı (H_TEMP), deadline baskısı (dp) yüksekken.

3.2 Adım 2 — Ekosistem Dinamiği: Kuralların Yarışı

Sezgi. Beş kuralın her birine bir “güç” değeri veririz: baskınlık vektörü $D(t)$. Her karar anında bu güçler güncellenir. Bir kuralın gücü; (a) o anki duruma ne kadar uyduğuna, (b) son zamanlarda ne kadar iyi iş çıkardığına ve (c) komşularıyla işbirliği/rekabetine göre artar veya azalır. Doğadaki *Lotka–Volterra* (avcı-av) rekabeti gibi: uygun olan güçlenir, uygunsuz olan bastırılır.

Önce **uyumluluk**: o anki işletim durumu c , her kuralın profili V_i ’ye ne kadar benziyor? Bunu kosinüs benzerliğiyle ölçeriz: $v_i(t_k) = \cos(V_i, c(t_k))$ (1’e yakın = çok uygun). Sonra iki seyrek matris kurallar arası etkileşimi yönetir: *işbirliği matrisi* A (bir kural güçlenince müttefikini de güçlendirir) ve *baskılama matrisi* S (bir kural güçlenince rakibini bastırır) — Tablo 2. Güncelleme:

$$D(t_{k+1}) = \text{normalize} \left(\underbrace{\alpha D(t_k)}_{\text{hafıza}} + \underbrace{\eta A D(t_k) - \lambda S D(t_k)}_{\text{işbirliği / baskılama}} + \underbrace{\beta \mathbf{p}(t_k)}_{\text{performans}} + \underbrace{\gamma \mathbf{v}(t_k)}_{\text{uyumluluk}} + \underbrace{\delta \mathbf{b}(t_k)}_{\text{arıza sıçraması}} \right). \quad (6)$$

Terimlerin anlamı, parantezlerde yazıldığı gibi: **hafıza** ($\alpha = 0.85$) gücün ani sıçramamasını sağlar (geçmiş unutmaz); **işbirliği/baskılama** ($\eta = \lambda = 0.12$) kuralları yarıştırır; **performans** $\mathbf{p} = (\text{CR} - \text{FR}) \cdot \mathbf{v}$ ($\beta = 0.25$) son zamanlarda işe yarayan kuralı ödüllendirir (tamamlama eksi başarısızlık); **uyumluluk** $\mathbf{v}$ ($\gamma = 0.20$) duruma uyan kuralı öne çıkarır; **arıza sıçraması** $\mathbf{b}$ ($\delta = 0.20$) bir robot bozulduğunda H_RECOV ve H_STAB kurallarını ani olarak yükseltir. Son adımda **normalize** tüm güçleri toplamı 1 olacak şekilde ölçekler (bir olasılık dağılımı gibi: $\|D\|_1 = 1, D \geq 0$).

Sezgi. $\eta AD - \lambda SD$ terimleri bu denklemi bir “kazanan güçlenir, rakibi zayıflar” yarışına çevirir — bu, çok-robotlu sistemlerde etkili olduğu gösterilen Lotka–Volterra tipi bir popülasyon dinamiğidir. Sistem atamayı çözmiyor; *hangi çözücünün* çalışacağını seçiyor.

Table 2: A (işbirliği) ve S (baskılama) matrislerinin sıfırdan-farklı girişleri. Diğer tüm girişler sıfır — yani etkileşim kasıtlı olarak seyrek ve yorumlanabilir tutulmuştur.

Matris	Kaynak	Hedef	Değer
A (işbirliği: güçlendirir)	H_CRIT	H_TEMP	0.20
	H_STAB	H_RECOV	0.20
S (baskılama: zayıflat)	H_TEMP	H_SPATIAL	0.30

Table 3: Beş hormon $\rightarrow$ beş atama kuralı.

Hormon	Kural (paradigma)	Ne yapar
H_SPATIAL	spatial_greedy	en yakın uygun robot + uymayanı reddet
H_CRIT	priority_first	önce yüksek öncelikli görevler
H_TEMP	edf_strict	en yakın deadline önce (varsayılan)
H_STAB	commit_once	kararı koru, değiştirme
H_RECOV	orphan_first	önce öksüz görevleri kurtar

3.3 Adım 3 — Seçici: Hangi Kural Çalışsın?

Sezgi. Yarışı her zaman beklemek tehlikeli olabilir: bir robot bozulduğunda “ekosistem yavaşça karar versin” diye beklersek görevler aksar. Bu yüzden iki *acil durum kuralı* (sert geçersiz-kılma) vardır; onlar tetiklenmezse normal yarışın kazanamı seçilir.

Seçici sırayla bakar:

1. Arıza oranı $c_4 > 0.05$ ise $\rightarrow$ doğrudan H_RECOV (arıza kurtarma). Beklemeden.
2. Deadline baskısı $c_3 > 0.5$ ise $\rightarrow$ doğrudan H_TEMP (deadline kuralı).
3. İkisi de yoksa $\rightarrow$ yarışın kazanamı: $p^*(t_k) = \arg \max_i D_i(t_k)$.

Eğer güçler birbirine çok yakınsa (belirgin bir kazanan yoksa), varsayılan olarak H_TEMP seçilir. Seçilen kural Tablo 3'teki mekanizmadır.

İş-koruma ilkesi (work conservation). Tüm kurallarda ortak bir altın kural: **yapılabilir iş varken hiçbir robot boş bırakılmaz.** Her karar anında her uygun robota o anki kuralın en iyi görevi verilir; bir robot bozulur veya takılırsa, onun görevleri (öksüzler) yeniden dağıtılır. Böylece bir aksaklık, *kahıncı görev kaybı* değil, en fazla bir *yeniden-deneme* maliyeti getirir. Bu, “kararı bir kez ver” (commit-once) yaklaşımının tam tersidir: o yaklaşım, bozulan robotun işlerini koşu bitene kadar atıl bırakır.

3.4 Adım 4 — Kural Kütüphanesi: Beş Atama Kuralı

Sezgi. Beş kural *aynı temel işlemi* kullanır: bir maliyet tablosu kurup en düşük toplam maliyetli eşleşmeyi bulmak (atama problemi). Farkları, maliyet tablosunu nasıl şekillendirdikleridir — yani neye öncelik verdikleri.

Ortak temel: uygunluk-maskeli bir maliyet matrisi $C^{(p)}$ verildiğinde, kural p şu *doğrusal toplam atama* (LSA, Macar algoritması) problemini çözer:

$$x^{(p)} = \arg \min_{x \in \mathcal{X}} \sum_{r \in \mathcal{R}} \sum_{\tau \in \mathcal{T}(t_k)} C_{r,\tau}^{(p)} x_{r,\tau}, \quad (7)$$

$\mathcal{X}$ kuralları zorlar: her robot en fazla Q görev ($\sum_{\tau} x_{r,\tau} \leq Q$), her görev en fazla bir robot ($\sum_r x_{r,\tau} \leq 1$) ve yetenek uygunluğu. Uygun olmayan çiftlerin maliyeti ∞ yapılır (maskeleme) — böylece o eşleşme asla seçilmez. $C_{r,\tau}$ Denk. (1)'nin temel maliyeti olmak üzere, beş kural şöyle ayrışır:

- **spatial_greedy** (H_SPATIAL): Maliyet yalnız yol süresidir ($C_{r,\tau}^{(p)} = T_{r,\tau}$). Her görevi en yakın uygun robota verir. *Ne zaman iyi*: yoğunlukta, hız önemliyken.
- **priority_first** (H_CRIT): Görevleri öncelik katmanlarına ($3 > 2 > 1$) böler ve önce yüksek öncelikli katmanı eşler; kritik görevler robotları ilk kapar. *Ne zaman iyi*: kritik görevler varken.
- **edf_strict** (H_TEMP, varsayılan): Görevleri en yakın deadline'a göre sıralar ve “zamanında varamayacak” eşleşmeleri sert biçimde eler (varış > deadline $\Rightarrow$ maliyet ∞). Ardından, deadline'ı bozmadan yolu kısaltan küçük bir *ucuz-ekleme* (cheapest-insertion) düzeltmesi uygular. *Ne zaman iyi*: deadline baskısı yüksekken (varsayılan kural).
- **commit_once** (H_STAB): Bir robotun kuyruğundaki görev yenilmez bir “sadakat bonusu” alır; atamalar dondurulur, gereksiz değişim (churn) sıfırlanır. *Ne zaman iyi*: kararlılık kritikken.
- **orphan_first** (H_RECOV): Bir robot bozulunca/takılınca öksüz kalan görevler, herkesten önce özel bir kurtarma turunda atanır. *Ne zaman iyi*: arıza anında (kurtarma süresini kısaltır).

Kurallar bağımsızdır (geçişte birbirine durum taşımazlar); tek kalıcı hafıza $D(t)$ 'dir. LSA en kötü durumda $O(\max(n, m)^3)$ 'tür; ama denenen ölçeklerde her karar milisaniye-altı sürer, yani kural seçimi gerçek-zaman bütçesini zorlamaz.

3.5 Tam Algoritma

Sezgi. Aşağıdaki döngü her karar anında bir kez çalışır: durumu oku $\rightarrow$ güçleri güncelle $\rightarrow$ kuralı seç $\rightarrow$ atamayı yap $\rightarrow$ kuyrukları yayınla.

Algorithm 1 AHE-MRTA Karar Döngüsü (her olayda bir kez)

Require: robotlar $\mathcal{R}$, açık görevler $\mathcal{T}(t_k)$, güçler $D(t_k)$, sabitler A, S, V , parametreler $\alpha, \eta, \lambda, \beta, \gamma, \delta$

- 1: $c \leftarrow \text{İŞLETİMDURUMVEKTÖRÜ}(\mathcal{R}, \mathcal{T}(t_k))$ $\triangleright$ 4 sayı: c_1 – c_4
- 2: $\mathbf{v} \leftarrow (\cos(V_i, c))_{i=1}^5$; $\mathbf{p} \leftarrow (\text{CR}_k - \text{FR}_k) \cdot \mathbf{v}$; $\mathbf{b} \leftarrow \text{ARIZASIÇRAMASI}(\mathcal{R})$ $\triangleright$ uyumluluk, performans, arıza
- 3: $D \leftarrow \text{normalize}(\text{clip}_{[0,1]}(\alpha D + \eta AD - \lambda SD + \beta \mathbf{p} + \gamma \mathbf{v} + \delta \mathbf{b}))$ $\triangleright$ güçleri güncelle
- 4: $p^* \leftarrow \text{ACILDURUMKURALI}(c)$ **varsa, yoksa** $\arg \max_i D_i$ $\triangleright$ kuralı seç
- 5: $x \leftarrow \text{KURAL}_{p^*}(\mathcal{R}, \mathcal{T}(t_k))$ $\triangleright$ atamayı çöz, Tablo 3
- 6: **yayınla** x 'ten robot-başına görev kuyrukları
- 7: **return** D, x

3.6 Kararlılık: Gereksiz Değişimi Önlemek

Sezgi. Her karar anında atamaları sıfırdan kurmak, robotları sürekli yön değiştirmeye iter (gerçek dünyada: Nav2 yolunu iptal et, baştan git — pahalı). Bu yüzden yeniden-planlama *minimal-bozulma* ilkesiyle yapılır: önceki plan korunur, yalnız gerçekten gerekli olduğunda bozulur.

Üç fren çalkantıyı (churn) bastırır:

- **Kapsamlı arıza-sadakati:** Arıza anında sadakat bonusu yalnız *öksüz* görevler için kalkar; sağlıklı atamalar yerinde kalır.
- **Yeniden-atama marj kapısı:** Bir görev, ancak yeni robotta tahmini varış eskisine göre *en az %10* iyileşirse taşınır. Muaf: öksüzler, kurtarmalar, deadline'ı çok yakın görevler.

- **Deadline-duyarlı önalım (preempt):** Takılan robotun kuyruğu toptan boşaltılmaz; yalnız beklemenin (tahmini 60 s) deadline'ı kaçıracağı görevler öksüz havuzuna düşer.

Bu paketin asıl faydası fiziksel (Gazebo/Nav2) yürütmede görülür: her yeniden-atama bir yol iptali + yeniden katetme demektir. (Sim, yeniden-atamayı bedelsiz modeller; bu yüzden buradaki kazanç simde görünmez, sahada görünür.)

3.7 Maliyet ve Sabit Ayarlar

Hesaplama maliyeti. LSA $O(\max(n, m)^3)$ 'tür; denenen ölçeklerde karar gecikmesi milisaniye-altı, ek düzeltme (swap-refine, ≤ 3 tur) ihmal edilebilir. İletişim ayak izi yalnız görev indeksleridir (~ 84 B/kuyruk); bellek ihmal edilebilir.

Tek-sefer ayarlanır, sonra dokunulmaz. Hormon-kural haritası (Tablo 3), A/S matrisleri (Tablo 2), profil tablosu V (Tablo 1) ve tüm sayısal parametreler ($\alpha=0.85$, $\eta=\lambda=0.12$, $\beta=0.25$, $\gamma=0.20$, $\delta=0.20$, softmax sıcaklığı $T=0.3$) *bir kez* belirlenir ve tüm senaryolarda, tüm ölçeklerde aynen kullanılır. Senaryoya göre ince ayar yok, öğrenme yok, uyarlanabilir parametre programı yok. **Bu önemli:** yöntemin gücü ince ayardan değil, doğru kuralı doğru anda seçmesinden gelir.

4 Somut Örnek: Bir Robot Bozulduğunda

Sezgi. Teoriyi tek bir anla bağlayalım: 5 robottan biri aniden bozuluyor (**robot_failure**). Sistem ne yapar?

1. **Durumu oku.** Arıza oranı $c_4 = 1/5 = 0.2$ olur (diğer sayılar anlık duruma göre).
2. **Güçleri güncelle.** Arıza sıçraması **b**, kurtarma kuralı H_RECOV 'un gücünü yükseltir; ayrıca V_{H_RECOV} 'un arıza boyutu 0.9 olduğundan uyumluluk $\cos(V_{H_RECOV}, c)$ de yüksektir.
3. **Kuralı seç.** $c_4 = 0.2 > 0.05$ acil durum kuralını tetikler: *yarıyı beklemeden* doğrudan $p^* = H_RECOV$.
4. **Atamayı yap.** **orphan_first:** bozulan robotun öksüz görevleri, sağlıklı robotlara özel bir kurtarma turuyla dağıtılır; ardından kalan açık görevler için normal deadline-bipartite çalışır.
5. **Yayınla.** Yeni kuyruklar yayınlanır; iş-koruma gereği hiçbir uygun robot boş kalmaz. Tamamlama sonuçları bir sonraki turda performans geri-beslemesini (**p**) besler.

İşte AHE-MRTA'nın dayanıklılığının özü budur: bir aksaklık kalıcı kayıp değil, en fazla bir yeniden-denemedir — ve sistem, bu anı algılayıp otomatik olarak “kurtarma moduna” geçer.

AHE-MRTA — Ecosystem-Driven Paradigm Selection (per-event cycle)

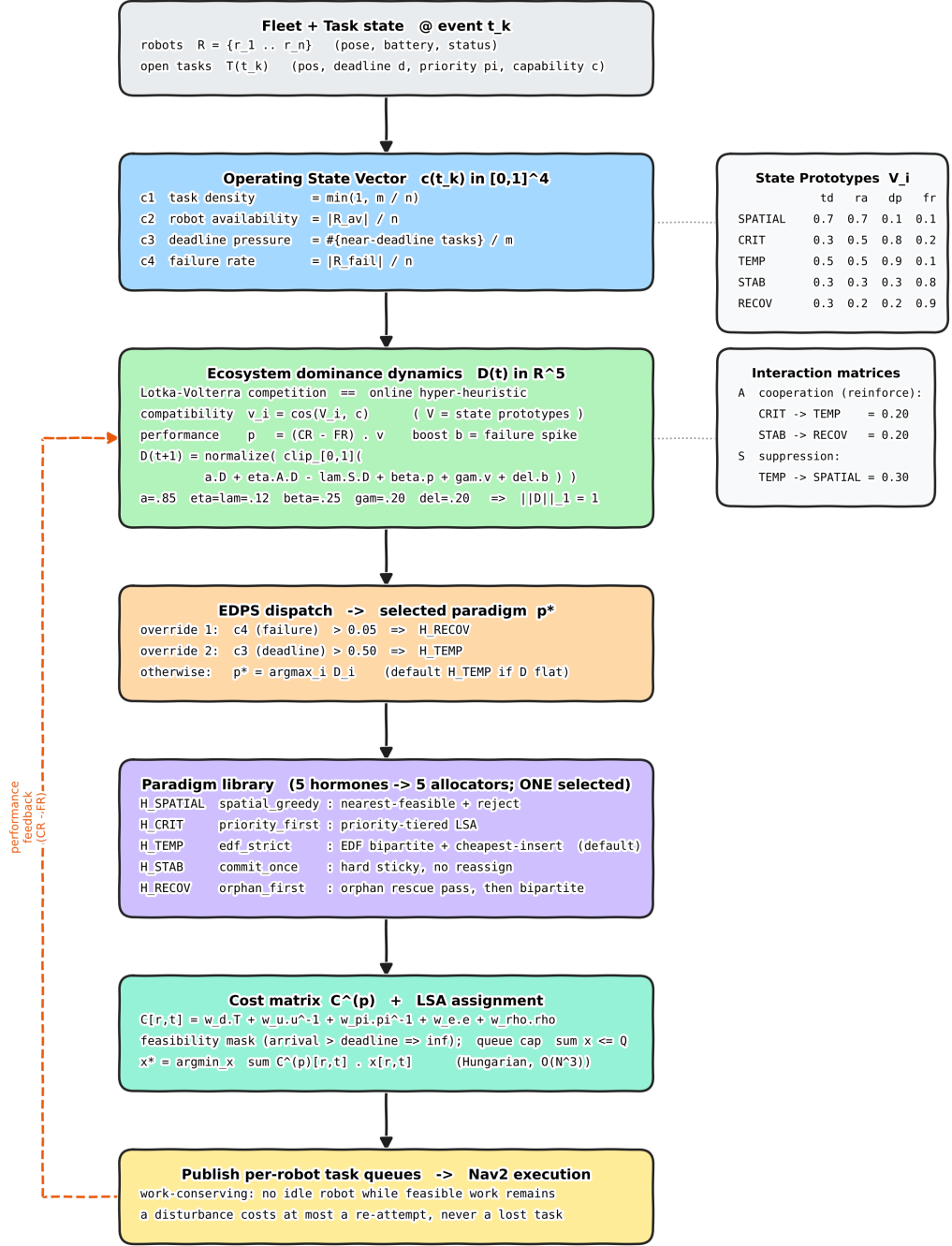


Figure 1: AHE-MRTA'nın bir karar anındaki (olay) akışı, yukarıdan aşağıya: filo ve görev durumu → dört sayılık *işletim durum vektörü* → beş hormonun yarıştığı *ekosistem dinamiği* (Denk. 6) → en güçlü hormonun seçilmesi ($p^* = \arg \max_i D_i$) → seçilen kuralın maliyet matrisini kurup atamayı (LSA) çözmesi → robot-başına görev kuyruklarının yayınlanması. Sol kenardaki turuncu döngü, tamamlanan/başarısız görevlerin bir sonraki kararda ekosisteme *geri-besleme* olarak dönmesidir.